

COMP 400 Project Report: InstantCurry Educational Proof Assistant

HYUNGKI KIM, McGill University (Undergraduate Student), Canada

JACOB ERRINGTON (SUPERVISOR), McGill University School of Computer Science (Faculty Lecturer), Canada

CCS Concepts: • **Theory of computation** → **Programming languages**; • **Logic** → **Equational reasoning and rewriting**.

ACM Reference Format:

Hyunki Kim and Jacob Errington (Supervisor). 2025. COMP 400 Project Report: InstantCurry Educational Proof Assistant.

1 Introduction

1.1 Motivation

The course COMP 302 is a required course for any computer science program. As a result, several hundreds of students usually take this course at a time. The course is split into two parts: one for learning concepts related to functional programming, and the other for more theoretical concepts in programming language theory and induction. In the former part, assignments are written in OCaml, using the Learn OCaml platform. [1] This allows for students to write their solutions in the browser, and have them automatically compiled, run, and graded. It can also provide immediate feedback on the mistakes in students' code. However for the second part, proofs have to be written and graded manually. This takes a large amount of time and effort on the part of the teaching staff, and does not allow for students to see feedback on their work in enough time for it to be most relevant for their learning.

The purpose of the InstantCurry (IC) project is to design and develop a simple proof assistant, along with a platform similar to Learn OCaml, that can formally verify and give feedback on inductive proofs given as assignments in this course.

1.2 History of the project

Several people have previously worked on this project over the last few years. In the last iteration, InstantCurry had the capabilities of parsing InstantCurry programs into an internal syntax, and translating them to be verified using the Rocq Prover. (formerly the Coq Proof Assistant) [2] This is a powerful research level proof assistant, which has been used to prove and verify important theorems in mathematics, such as the Four Color Theorem, [3] as well as to build, certify, and verify compilers used in the industry. [4], [5] However, Rocq is very complex and not necessarily user-friendly. It is very much overpowered for the simple proofs that are required in the course, and its complexities and idiosyncrasies caused difficulties with translating certain types of proofs. For example, InstantCurry was unable to prove the the length of a list was preserved by the map function. Furthermore, The Rocq backend was opaque and didn't provide friendly error messages, nor allowed for easily inspecting partially parsed or proved programs, which is essential for knowing at which point the program failed and providing helpful error messages and suggestions.

To that end, an effort to migrate InstantCurry away from Rocq had been made, including partial typechecking of the internal syntax, and provide some facilities in evaluating IC expressions.

1.3 Our contributions

We completed the typechecker, and implemented a native proof checking system in OCaml. This involved polymorphic typechecking and inference, as well as a system for defining constructs, stating and using previously proved theorems, and checking each individual proof step against valid deduction rules. The project is mainly focused on checking proofs by induction on lists, thus we also implemented a mechanism for stating and verifying inductive hypotheses as well as base cases and inductive steps.

2 InstantCurry

InstantCurry consists of a few main parts. First, there is the proof language. That is, the language that students write the proofs in. This is parsed using OCamllex [6] and Menhir, [7] an LR(1) parser generator written in OCaml. This is then parsed into a parse tree, which is scopechecked and lifted to an internal AST. This includes distinguishing between bound, free, and universally quantified (meta) variables. The AST then gets typechecked before passing through the proof checker. There is also a separate web interface portion of the project, providing an easy way for students to write IC proofs, and to provide feedback.

2.1 Syntax

The proof language is fairly simple. It consists of statements, marked by specific tokens. The definitions, arguments, and proof terms are basically just notated in OCaml.

Below is an example of an IC program including some definitions and a theorem with proof that the map function preserves the length of a list.

```

DEFINITION.
let rec len (l : 'a list) : nat =
  match l with
  | [] => 0
  | x :: xs => 1 + len xs
end

DEFINITION.
let rec map (f : 'a -> 'b) (l : 'a list) : 'b list =
  match l with
  | [] => []
  | x :: xs => f x :: (map f xs)
end

THEOREM (map_len).
FORALL (l : 'a list) (f : 'a -> 'b) :
  len (map f l) = len l : nat.

PROOF.
BY INDUCTION ON l.

CASE l = [].

```

```

WTS: len (map f []) = len [].

LHS = len (map f []).
      = len []                      -- BY defn.

RHS = len [].

CASE l = x :: xs.
IH: len (map f xs) = len xs.
WTS: len (map f (x :: xs)) = len (x :: xs).

LHS = len (map f (x :: xs)).
      = len (f x :: map f xs)      -- BY defn.
      = 1 + len (map f xs)         -- BY defn.
      = 1 + len xs                  -- BY IH.

RHS = len (x :: xs).
      = 1 + len xs                  -- BY defn.

QED.

```

For comparison, this is an excerpt of the proof syntax from an example given to students in class. Normally, these proofs are done by students on paper. Note the closeness in syntax, meaning students can jump right into writing proofs in IC.

```

THEOREM. (Reverse is a self-inverse.)
  rev (rev l) = l

PROOF. By induction on l.

CASE l = [].
  WTS: rev (rev []) = []

  LHS = rev (rev [])
        = rev [] -- by def
        = [] -- by def
        = RHS

CASE l = x::xs.
  WTS: rev (rev (x::xs)) = x :: xs
  IH: rev (rev xs) = xs

  LHS = rev (rev (x::xs))
        = rev (rev xs @ [x]) -- by def
        = rev (x::[]) @ rev (rev xs) -- by lemma
        = rev [x] @ xs -- by IH
        = (x::[]) @ xs -- by def (many times)
        = x :: xs -- by def some more

```

```

      = RHS
QED.

```

In the step case, several 'by definition' steps were collapsed into a single step. This is fine, but you need to be sure you're right. It never hurts to write out all the steps.

One thing to note is that in InstantCurry, every proof step only expects one transformation, so that each instance of the definition application must be explicitly stated.

2.2 Proof Checking

After typechecking, the AST is passed through the proof checker. It builds up an environment of the definitions and proved theorems. Theorems are stated along with a list of universally quantified variables. In IC, theorems are mainly inductive over lists, and proofs begin by indicating the induction variable. This is followed by the base case and the inductive step. Each case has a WTS, or a “want-to-show” statement, which is the statement to be proved for that case. The inductive step additionally comes with the inductive hypothesis statement. For each case, the left-hand side and the right-hand side of the statement to prove is brought to equality by taking a series of deductive steps.

The WTS and the inductive hypotheses are not required to be explicitly stated, as InstantCurry can infer these from the theorem statement. This also allows for IC verify the students' WTS and IH, if given. For each case, the variable of induction is substituted in the theorem statement with either an empty list, or an arbitrary list with pattern ``x::xs``.

Likewise for the inductive hypothesis, by substituting just the tail of the list ``xs``.

```

Given x : 'a list and theorem statement  $\forall x, T$ 

Case x = []      => [[]/x] T.
Case x = y :: ys => [(y :: ys)/x] T.
IH              => [ys/x] T.

```

For every deductive step, IC looks for a valid substitution of the expression that makes it match the next, based on the given justification.

```

<justification> ::= -- BY defn.
                  | defn of <variable>.    # function name
                  | IH.
                  | <variable>.             # theorem name
                  | commonsense.
                  | eval.

```

Definitions in IC are parsed into closures with nested lambda abstractions. When checking a definition, IC finds all instances of function calls, and tries substituting each one with the definition, and applying it to its arguments. Justifications `--by defn of <name>` can be stated with a particular function name or omitted.

```

len (f x :: map f xs)

=> (fun l ->
    match l with
    | [] => 0

```

```

    | x :: xs => 1 + len xs)
  (f x :: map f xs)

= 1 + len (map f xs)          -- BY defn (of len).

```

Justifications --by IH (Inductive Hypothesis) and theorems are checked in both directions of the equation by trying to find a subexpression that matches one side and substituting it by the other.

```

IH: len (map f xs) = len xs.

    1 + len (map f xs)
  = 1 + len xs          -- BY IH.

```

Of note is that IC will recursively check through subtrees of the expressions. Here, the IH was applied at the subtree ``1 + _``.

Justifications --by commonsense encapsulates basic properties of arithmetic, such as the additive identity, commutativity and associativity of addition and multiplication, and associativity of function composition.

```

    x + sum xs + acc
  = sum xs + x + acc    -- BY commonsense (commutativity).
  = sum xs + (x + acc)  -- BY commonsense (associativity).

```

Justification --by eval is used when evaluating a lambda abstraction that may be present in a theorem statement.

```

THEOREM (map_fusion).
FORALL (f : 'b -> 'c) (g : 'a -> 'b) (l : 'a list) :
  map (fun (x : 'a) => f (g x) end) l = map f (map g l) : 'c list.

LHS = map (fun (x : 'a) => f (g x) end) (x :: xs).
    = (fun (x : 'a) => f (g x) end) x ::
      map (fun (x : 'a) => f (g x) end) xs          -- BY defn of map.
    = f (g x) :: map (fun (x : 'a) => f (g x) end) xs -- BY eval (head).

```

2.3 Types

InstantCurry has 2 inductive types, natural numbers and lists, as well as function types. It is based on the Hindley Milner type system, including parametric polymorphism. Thus type annotations including type variables, or even omitting annotations altogether is possible. However, all type annotations are required for theorem statements for pedagogical reasons.

```
Types A, B ::= Nat | List A | A -> B
```

IC uses Algorithm W to infer types through unification.

Type inference for definitions is the same as it is in OCaml.

Proofs, cases, WTS, IH, and proof steps are all inferred based on the parent theorem statement.

The induction variable is expected to be in the theorem context as a universally quantified variable

It is also expected to be an inductive type.

```

THEOREM (theorem_name).
FORALL (l : 'a list) (x_1 : t_1) ... (x_n : t_n) :
  left = right : t.

```

```
PROOF.
BY INDUCTION ON l ('a list).
```

The case statement then binds variables as needed. For lists, this must be in a cons pattern form.

```
CASE l = x::xs => x : 'a', xs : 'a list
WTS: left = right : t
IH: left = right : t
```

The inferred types for the WTS, IH, and each proof step is checked against that of the theorem statement.

```
LHS = e : t
     = e_1 : t
     ...
     = e_n : t

RHS = e' : t
     = e'_1 : t
     ...
     = e'_n = e_n : t
QED.
```

2.4 Error Messages

One of the goals of InstantCurry is to provide friendly error messages with helpful advice. Currently, this is implemented for syntax errors.

Menhir is a shift reduce parser. It shifts tokens onto a stack as it reads the input, and reduces it when it is able to complete a grammar rule. Each of these actions is a state transition of the state machine. With Menhir's newer incremental API, [7] it is able to enumerate all error states to a file, in which we can write corresponding error messages.

```
program: THEOREM LPAR IDENT RPAR SEP WTS
##
## Ends in an error in state: 5.
##
## stmt -> THEOREM LPAR IDENT RPAR SEP . FORALL nonempty_list(strict_arg) COLON
eqn COLON ty SEP proof [ THEOREM PRINT EOF DEFINITION CONST ]
##
## The known suffix of the stack is as follows:
## THEOREM LPAR IDENT RPAR SEP
##

I expected a 'FORALL' here, but instead I found '$0'.
Hint: Theorems should be stated like this:
  THEOREM (theorem_name).
  FORALL (arg_1 : type_1) ... (arg_n : type_n) : lhs = rhs : expr_type.
  PROOF.
  ...
```

The program line just shows one possible way to get to the error state. The production rule under consideration is shown, as well as the current stack of tokens followed by valid continuations.

The API gives access to the token stack, which we can refer to in the error message. The error messages gets compiled to a function that maps states to messages. We provide the error messages to the frontend UI in JSON format.

3 Conclusion

3.1 Related work

Previous work was done for the first part of the COMP 302 course, in extending the Learn OCaml platform to facilitate automated grading. [8] The standard grading system focuses mostly on the correctness of the output. Hameer et al. used the AST generated from the students' code to perform deeper inspection, such as whether certain language constructs were used. This allowed for more specific feedback, enhancing the learning experience. They reported that it was not feasible to write graders for assignments every semester, and that a repository was built up for reuse. Given the relatively constrained proof environment and solutions in InstantCurry, we expect that building up model solutions would be a simple task. Furthermore, this would allow comparison of the students' solutions to the model solutions, providing opportunities for generating more useful and specific feedback.

Other mainstream proof assistants, such as Rocq and Isabelle are certainly more powerful, but are too complex for the scope of our use case, and the learning curve for students would be too high. On the other hand, the InstantCurry proof syntax is built on OCaml, which students will already be familiar with, and follows closely the syntax taught in class for writing proofs on paper.

There are other proof assistants developed to be educational tools rather than general purpose theorem provers, with the goal of being friendly and giving feedback to guide the students. For example, Emmy[9] and ProofBlocks.[10] However, Emmy is designed for a general logic course, and the syntax is much closer to that of standard first-order logic, mixed with that of a typical imperative programming language. Meanwhile, InstantCurry's syntax is a very natural progression for students in this course. ProofBlocks takes a different approach. It presents students with a theorem, and several 'blocks' of the steps of the proof, in a random order. The student must then rearrange the blocks to form the complete proof. While this does allow for ease of providing feedback, due to the students' options being constrained, this does not quite align with the goals of IC. Furthermore, ProofBlocks is focused more on proofs in discrete math.

4 Future work

There are several ways IC can be enhanced further to support student learning. As IC becomes used in class, we can collect students' solutions and attempts, and analyze what the most common mistakes and misunderstandings are, allowing us to effectively target them with the feedback system as well as in lectures.

Work remains to be done to fully meet the goals of the project. At the moment, there are no helpful error messages with advice for scoping errors and type errors. This would involve attaching information, such as source location, to the AST for each token. It is also missing tree types and custom types for students to define their own syntax trees, so that proofs on syntax trees can be handled by IC.

Beyond the main goals of IC, there are other ways to further extend the project. The Haskell language features "Typed Holes", where programs can be partially written with "holes" left in the code. The compiler can then infer the type that the hole should have, and provide the user with all identifiers in scope that has matching types. Similarly, IC could allow for partial proof steps and suggest how to fill the hole.

A possible development is to show, side-by-side, an algebraic representation of the statements and deduction steps. This would help students to abstract away the object level semantics of the statements being manipulated, and think at the syntactic proof manipulations. It remains to be seen how much of a problem this is for students.

Bibliography

- [1] OCaml Software Foundation, “Learn OCaml.” [Online]. Available: <https://ocaml-sf.org/learn-ocaml/>
- [2] “The Rocq Prover.” [Online]. Available: <https://rocq-prover.org/>
- [3] Georges Gonthier, “A computer-checked proof of the Four Color Theorem,” *Microsoft Research Cambridge*, 2005.
- [4] Xavier Leroy, “Formal Certification of a Compiler Back-end or: Programming a Compiler with a Proof Assistant,” in *Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, Jan. 2006. doi: 10.1145/1111037.1111042.
- [5] Xavier Leroy, “Formal verification of a realistic compiler,” *Commun. ACM*, vol. 52, July 2009, doi: 10.1145/1538788.1538814.
- [6] “Parsing with OCamllex and Menhir.” [Online]. Available: <https://dev.realworldocaml.org/parsing-with-ocamllex-and-menhir.html>
- [7] François Pottier and Yann Régis-Gianas, “The Menhir Parser Reference Manual.” [Online]. Available: <https://gallium.inria.fr/~fpottier/menhir/manual.html>
- [8] Aliya Hameer and Brigitte Pientka, “Teaching the Art of Functional Programming Using Automated Grading (Experience Report),” in *Proc. ACM Program. Lang.*, Jan. 2018.
- [9] Junfeng Xu, “Emmy: A Proof Assistant for Reasoning about Programs,” June 2019.
- [10] Seth Poulsen, Mahesh Viswanathan, Geoffrey Herman, and Matthew West, “Proof Blocks: Autogradable Scaffolding Activities for Learning to Write Proofs,” in *Proceedings of the 27th ACM Conference on on Innovation and Technology in Computer Science Education*, 2022.